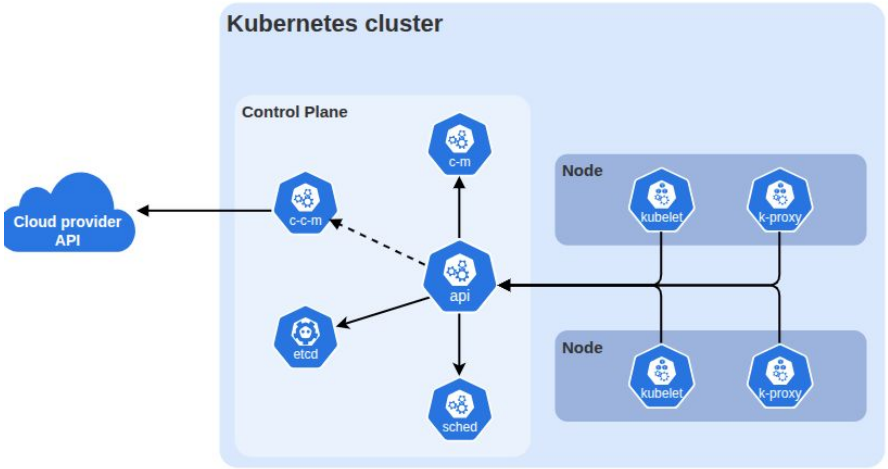
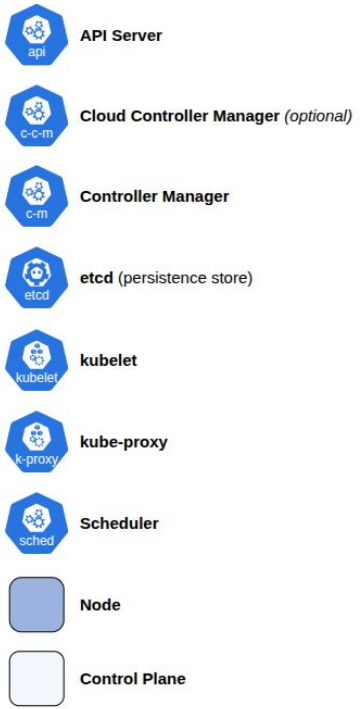
Gerência de configuração

Continuação ao Kubernetes

Prof. Diego Cardoso Borda Castro

Componentes para K8s





Cluster K8s

- K3D
  - K3S
  - IOT
- KubeAdm
  - Administrado pela própria equipe do kubernetes
  - Maneira de criar um cluster de forma on-premise
  - Maior trabalho
  - VM física servindo como

Azure VMs

Você não tem uma assinatura? Confira as opções a seguir.



Começar com uma avaliação gratuita do Azure

Receba um crédito gratuito de 200 dólares para produtos e serviços do Azure, além de 12 meses de serviços gratuitos populares.

Iniciar



Gerenciar o Microsoft Entra ID

Gerencie o acesso, defina políticas inteligentes e aprimora a segurança com o Microsoft Entra ID.

Exibir Saiba mais



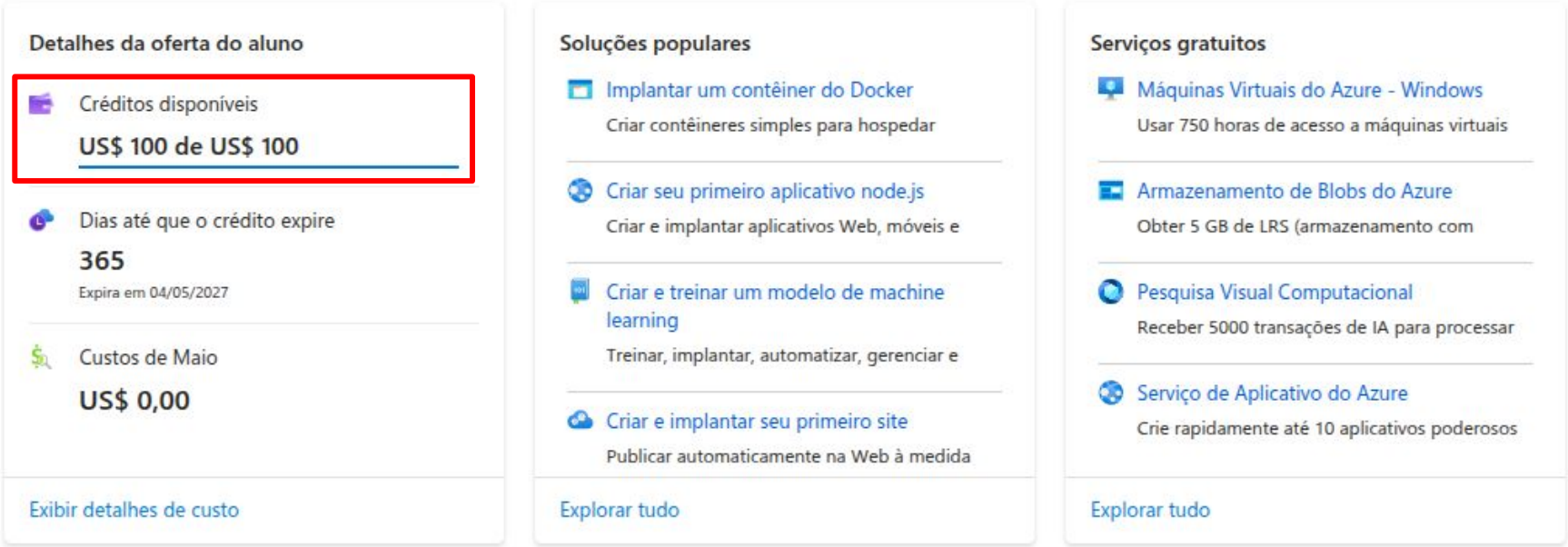
Microsoft Azure for Students

Receba software gratuito, crédito do Azure ou acesso ao Azure Dev Tools for Teaching após verificar seu status acadêmico.

Iniciar

Servicos do Azure

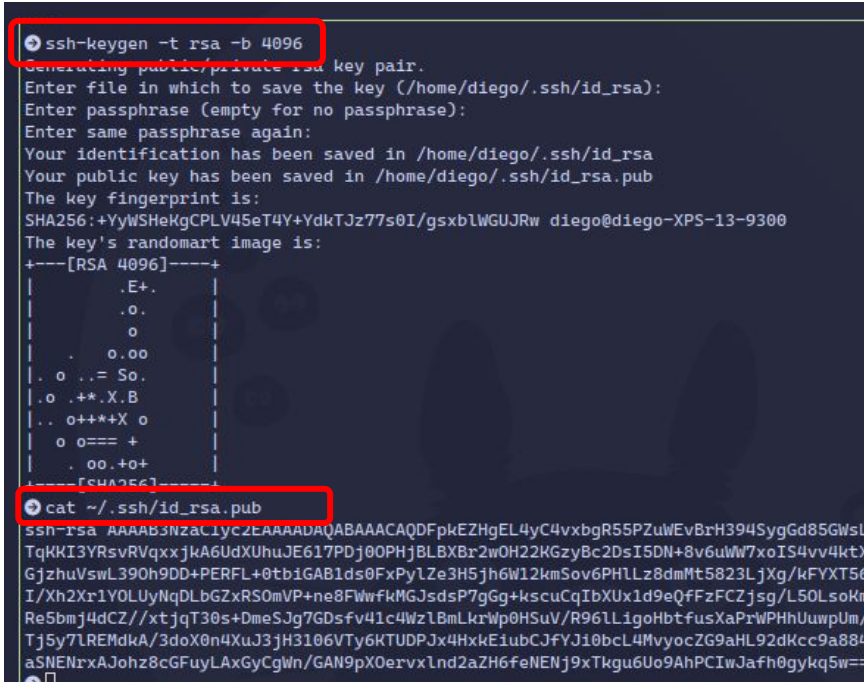
Azure VMs


Detalhes da oferta do aluno
Créditos disponíveis
US$ 100 de US$ 100
Dias até que o crédito expire
365
Expira em 04/05/2027
Custos de Maio
US$ 0,00
Exibir detalhes de custo
Soluções populares
Implantar um contêiner do Docker
Criar contêineres simples para hospedar
Criar seu primeiro aplicativo node.js
Criar e implantar aplicativos Web, móveis e
Criar e treinar um modelo de machine learning
Treinar, implantar, automatizar, gerenciar e
Criar e implantar seu primeiro site
Publicar automaticamente na Web à medida
Explorar tudo
Serviços gratuitos
Máquinas Virtuais do Azure - Windows
Usar 750 horas de acesso a máquinas virtuais
Armazenamento de Blobs do Azure
Obter 5 GB de LRS (armazenamento com
Pesquisa Visual Computacional
Receber 5000 transações de IA para processar
Serviço de Aplicativo do Azure
Crie rapidamente até 10 aplicativos poderosos
Explorar tudo

Azure VMs

- https://portal.azure.com
  - Instalando o cliente

https://learn.microsoft.com/pt-br/cli/azure/install-azure-cli-linux?view=azure-cli-latest&pivots=apt



Configurando os nodes

```
az group create --name rg-student --location eastus
```

```
az group create --name rg-student --location eastus
```

```
az vm create \
  --resource-group rg-student \
  --name vm1 \
  --image Ubuntu2204 \
  --size Standard_D2s_v3 \
  --location centralus \
  --admin-username azureuser \
  --ssh-key-values ~/.ssh/id_rsa.pub \
  --public-ip-sku Standard
```

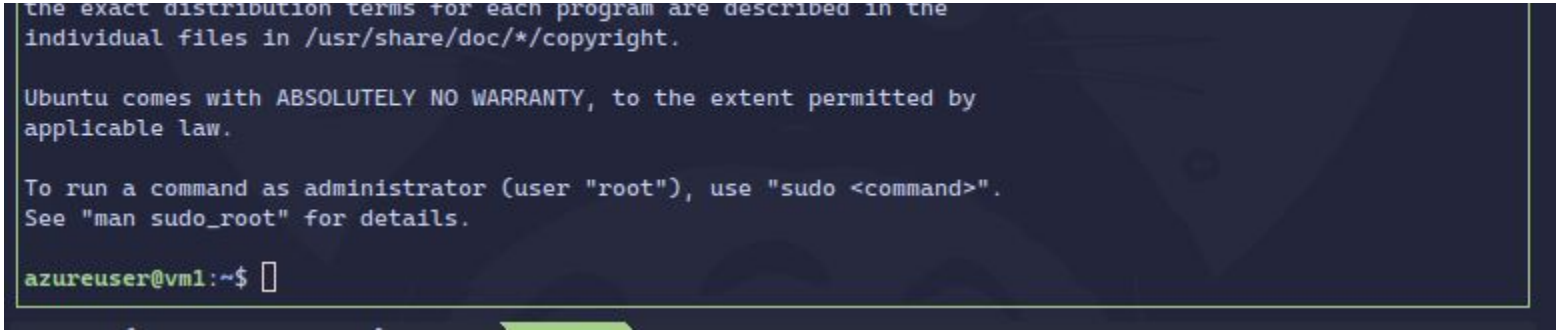
Nome ↑	Assinatura	Grupo de Recursos	Localização	Status	Sistema operacio...
vm1 ···	Azure for Students	rg-student	Central US	Em execução	Linux

```
~
az vm create   --resource-group rg-student   --name vm1   --image Ubuntu2204   --size Standard_D2s_v3   --location centralus   --admin-username azureuser   --ssh-key-values ~/.ssh/id_rsa.pub   --public-ip-sku Standard
The default value of '--size' will be changed to 'Standard_D2s_v5' from 'Standard_DS1_v2' in a future release.
Consider upgrading security for your workloads using Azure Trusted Launch VMs. To know more about Trusted Launch, please visit https://aka.ms/TrustedLaunch.
{
  "fqdns": "",
  "id": "/subscriptions/1f1ce909-7e19-4a46-9206-0451240535e7/resourceGroups/rg-student/providers/Microsoft.Compute/virtualMachines/vm1",
  "location": "centralus",
  "macAddress": "38-33-C5-D5-BA-17",
  "powerState": "VM running",
  "privateIpAddress": "10.0.0.4",
  "publicIpAddress": "135.119.24.221",
  "resourceGroup": "rg-student"
}
az vm list-ip-addresses -o table
VirtualMachine    PublicIPAddresses    PrivateIPAddresses
----------------  -------------------  --------------------
vm1               135.119.24.221       10.0.0.4
az vm show -d -g rg-student -n vm1 --query publicIps -o tsv
135.119.24.221
```

Configurando os nodes

```
az vm open-port \
  --resource-group rg-student \
  --name vm1 \
  --port 22
```

```
ssh -i ~/.ssh/id_rsa azureuser@135.119.24.221
```



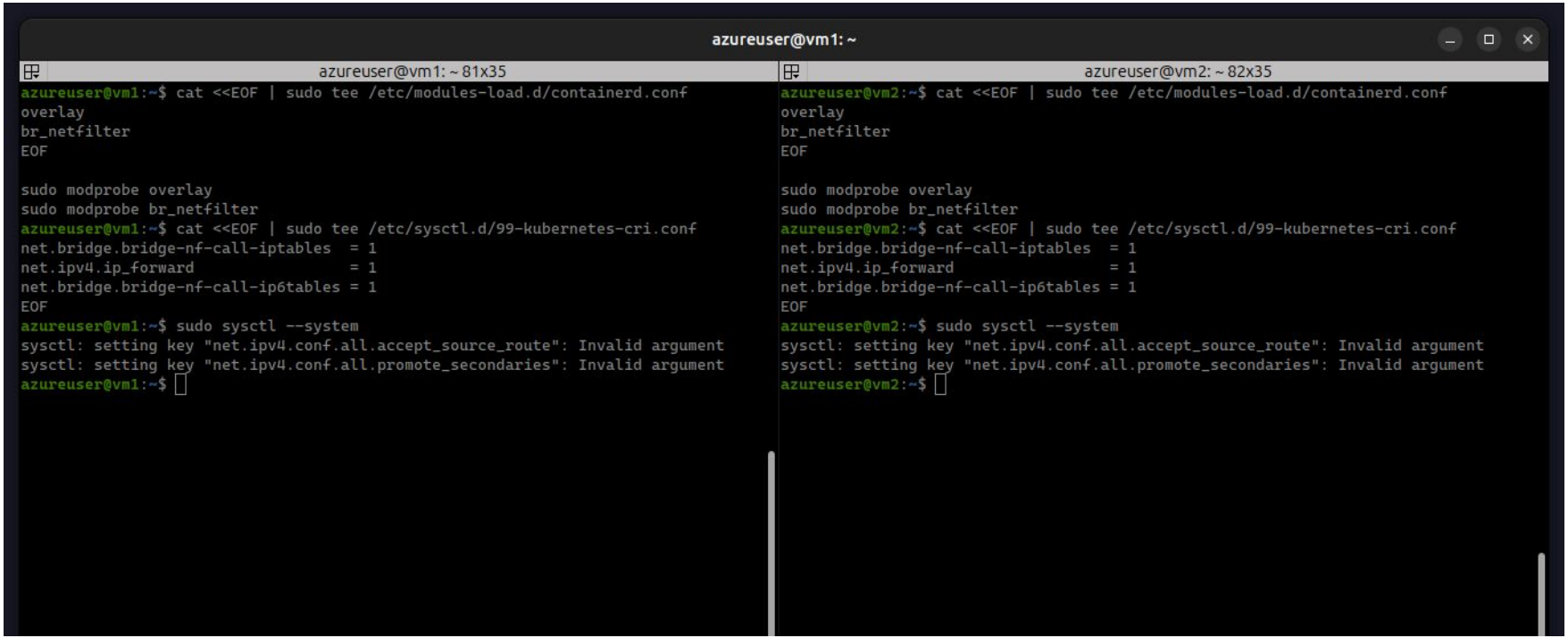
Configurando os nodes

- Configurar
  - overlay
  - netfilter
  - containerD
  - kubelet
  - kubeadm
  - kubectl

```
cat <<EOF | sudo tee
/etc/modules-load.d/containerd.conf
overlay
br_netfilter
EOF

sudo modprobe overlay
sudo modprobe br_netfilter
```

Configurando os nodes



```
sudo apt-get update

sudo apt-get install ca-certificates curl gnupg --yes

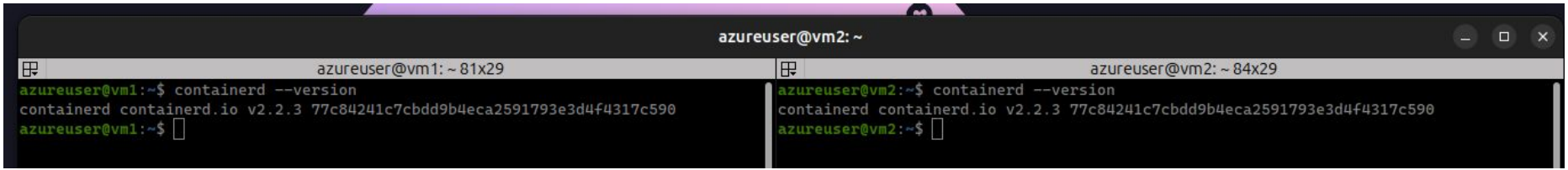
sudo install -m 0755 -d /etc/apt/keyrings

curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o
/etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg

echo \
        "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg]
      https://download.docker.com/linux/ubuntu \
        $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
        sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

Configurando os nodes

```
sudo apt update && sudo apt install containerd.io -y
```




```
sudo mkdir -p /etc/containerd && containerd config default | sudo tee /etc/containerd/config.toml

sed -i 's/SystemdCgroup = false/SystemdCgroup = true/g' /etc/containerd/config.toml

sudo systemctl restart containerd
```

Introdução

```
sudo apt-get update && \
sudo apt-get install -y apt-transport-https ca-certificates curl

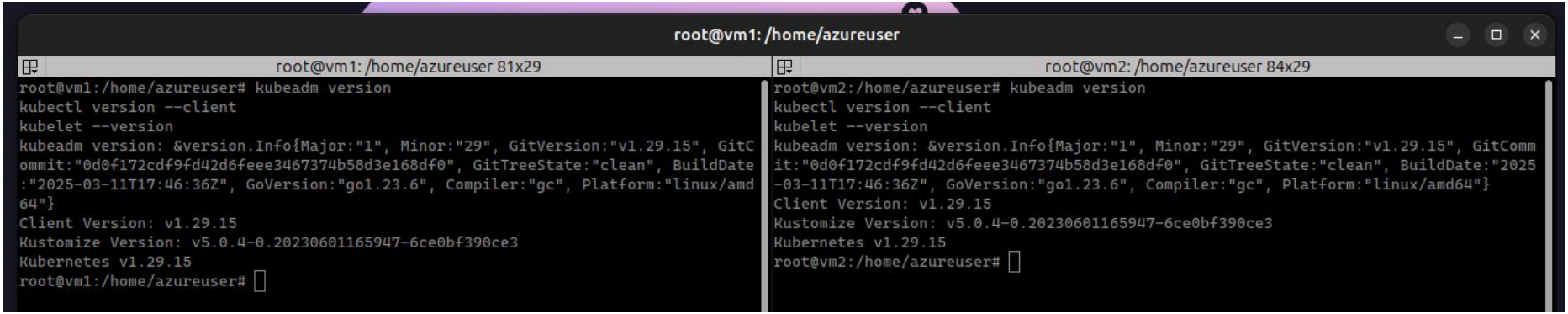
sudo mkdir -p /etc/apt/keyrings

curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.29/deb/Release.key \
  | sudo gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg
```

```
echo "deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] \
https://pkgs.k8s.io/core:/stable:/v1.29/deb/ /" \
| sudo tee /etc/apt/sources.list.d/kubernetes.list
```

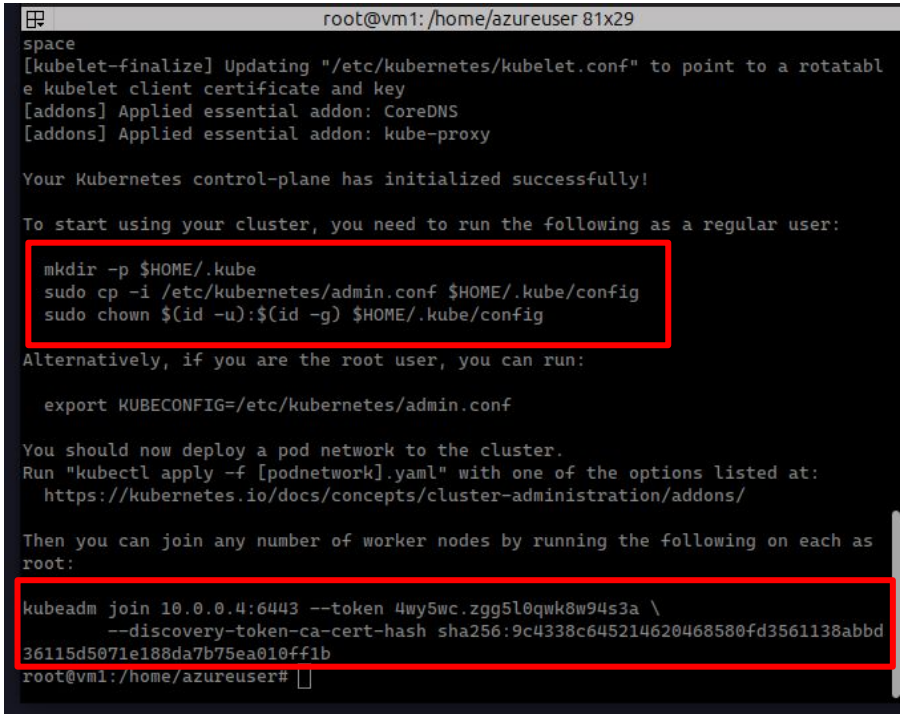
Configurando os nodes

```
sudo apt update
sudo apt install -y kubelet kubeadm kubectl
```

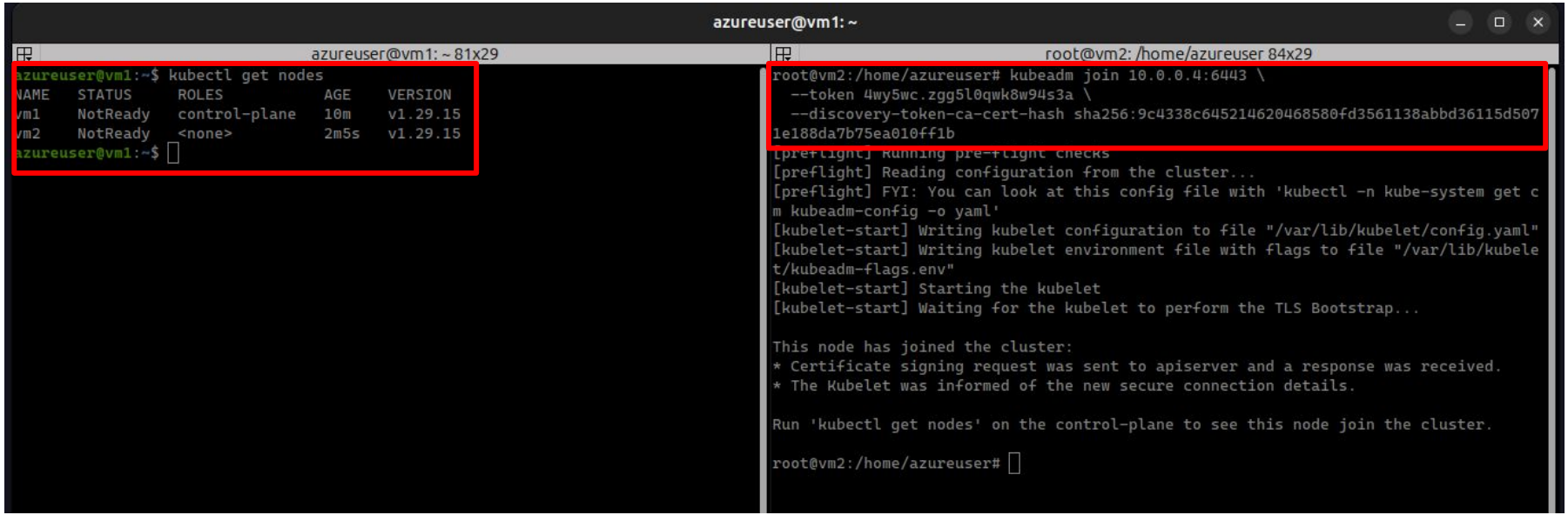


Configurando os Control Plane

- Configurar
  - kubeadm init
  - Adicionar os nodes

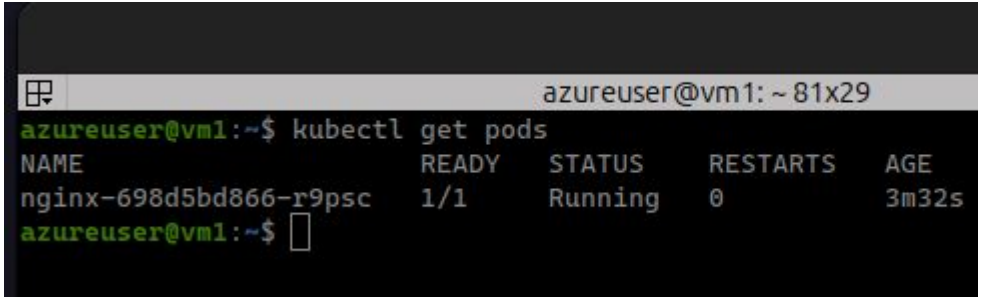


Configurando os Control Plane



Configurando os Control Plane

- Configurar
  - Container Network Interface
    - Calico
    - Flannel
    - …

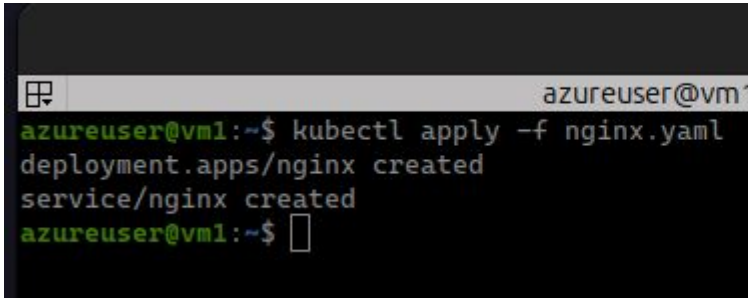


kubectl apply -f https://raw.githubusercontent.com/projectcalico/calico/v3.26.0/manifests/calico.yaml

Configurando os Control Plane

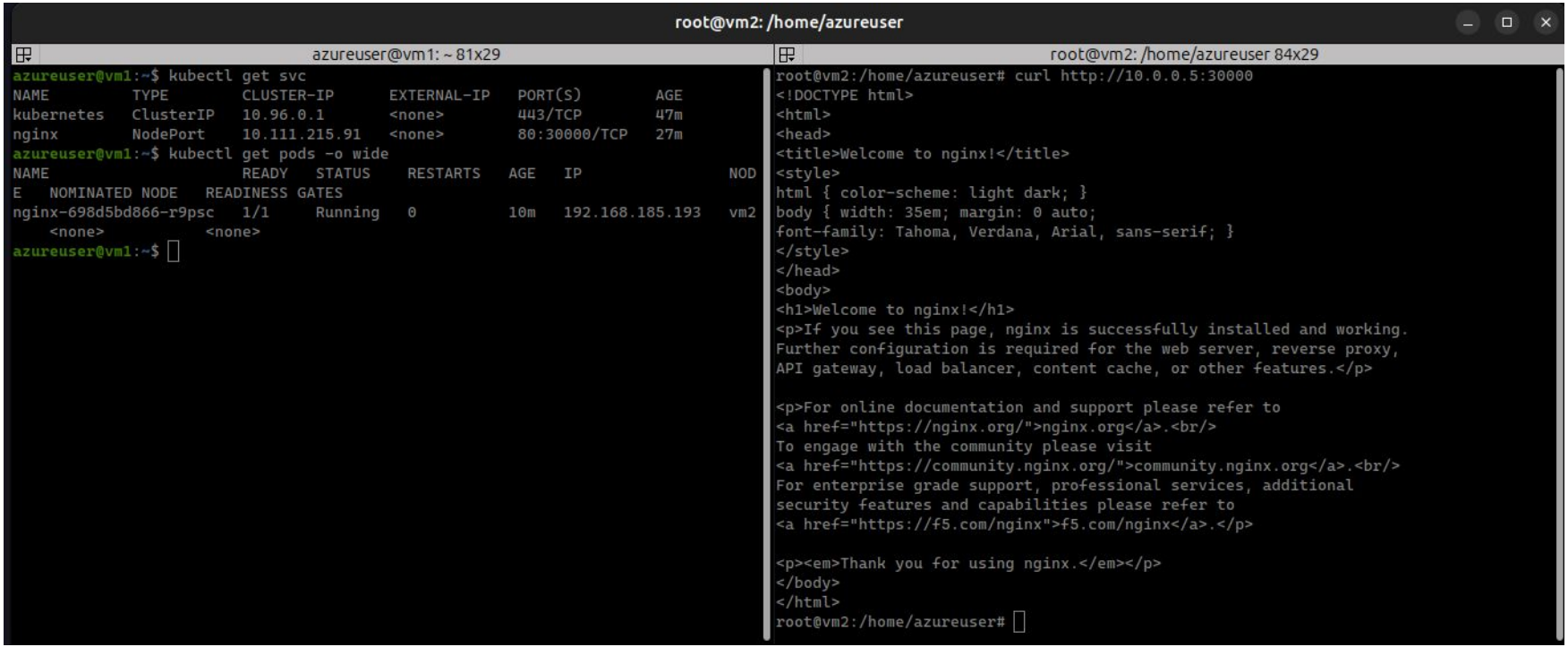
```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx
spec:
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx
        ports:
        - containerPort: 80
```

```yaml
apiVersion: v1
kind: Service
metadata:
  name: nginx
spec:
  selector:
    app: nginx
  ports:
  - port: 80
    targetPort: 80
    nodePort: 30000
  type: NodePort
```



- Criar arquivo no servidor
  - Aplicar o arquivo

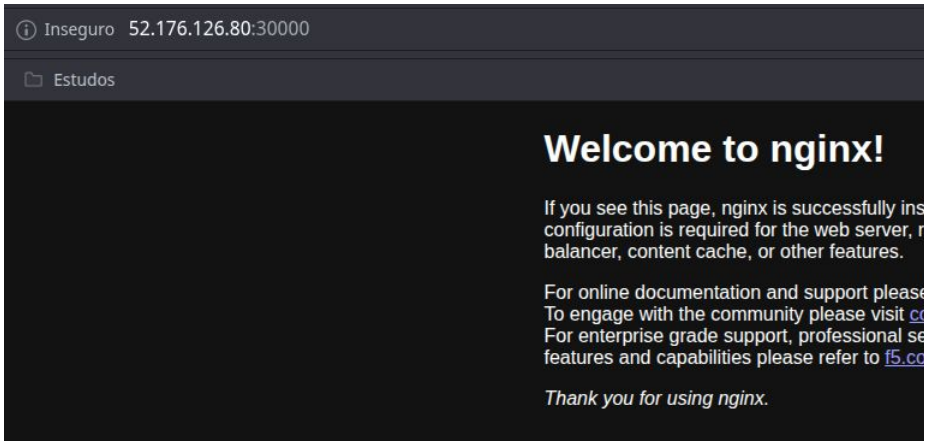
Configurando os Control Plane

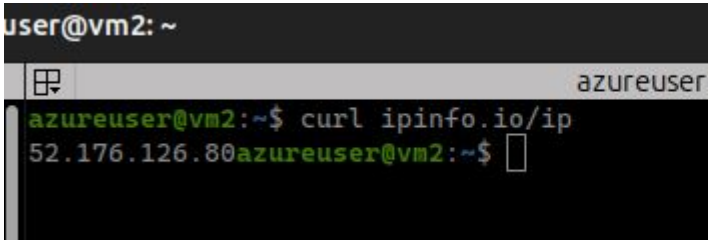


Configurando os Control Plane

- Libera a porta externa
  - Acessa pelo ip externo

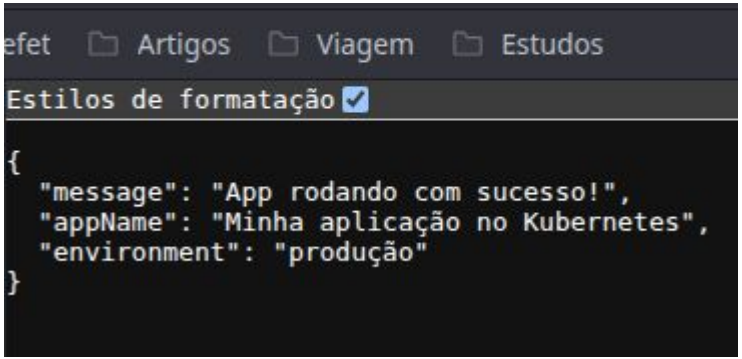
```
az vm open-port --resource-group rg-student --name vm2 --port 30000 --priority 1010
```





Variáveis

- Passagem de variáveis pra dentro do pod




```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: node-env-demo
spec:
  replicas: 1
  selector:
    matchLabels:
      app: node-env-demo
  template:
    metadata:
      labels:
        app: node-env-demo
    spec:
      containers:
        - name: node-app
          image: diegocbcastroo/node-env-var:latest
          ports:
            - containerPort: 3000
          env:
            - name: APP_NAME
              value: "Minha aplicação no Kubernetes"
            - name: ENVIRONMENT
              value: "produção"
```

Variáveis

- ConfigMap
  - chave : valor
  - kubectl edit configmap app-config
  - kubectl rollout restart deployment node-env-demo

```
kubectl create configmap app-config \
  --from-literal=APP_NAME="App via ConfigMap" \
  --from-literal=ENVIRONMENT="dev"
configmap/app-config created
kubectl get configmap
NAME               DATA   AGE
app-config         2      34s
kube-root-ca.crt   1      33m
kubectl describe configmap app-config
Name:         app-config
Namespace:    default
Labels:       <none>
Annotations:  <none>

Data
====
APP_NAME:
----
App via ConfigMap

ENVIRONMENT:
----
dev
```

Variáveis

- ConfigMap
  - kubectl delete configmap app-config
  - kubectl create configmap app-config-file \
    --from-file=APP_NAME

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config-yaml
data:
  APP_NAME: "App via YAML"
  ENVIRONMENT: "homologacao"
```

Variáveis

- Secrets
  - Secret no Kubernetes NÃO é criptografia forte por padrão
    - echo "YWRtaW4=" | base64 -d

```
kubectl create secret generic app-secret \
  --from-literal=DB_USER=admin \
  --from-literal=DB_PASS=123456
secret/app-secret created
kubectl get secret app-secret -o yaml
apiVersion: v1
data:
  DB_PASS: MTIzNDU2
  DB_USER: YWRtaW4=
kind: Secret
metadata:
  creationTimestamp: "2026-05-05T06:20:36Z"
  name: app-secret
  namespace: default
  resourceVersion: "2283"
  uid: eddcdb1b-694a-40aa-84b4-9f7b106ae1fb
type: Opaque
```

Variáveis

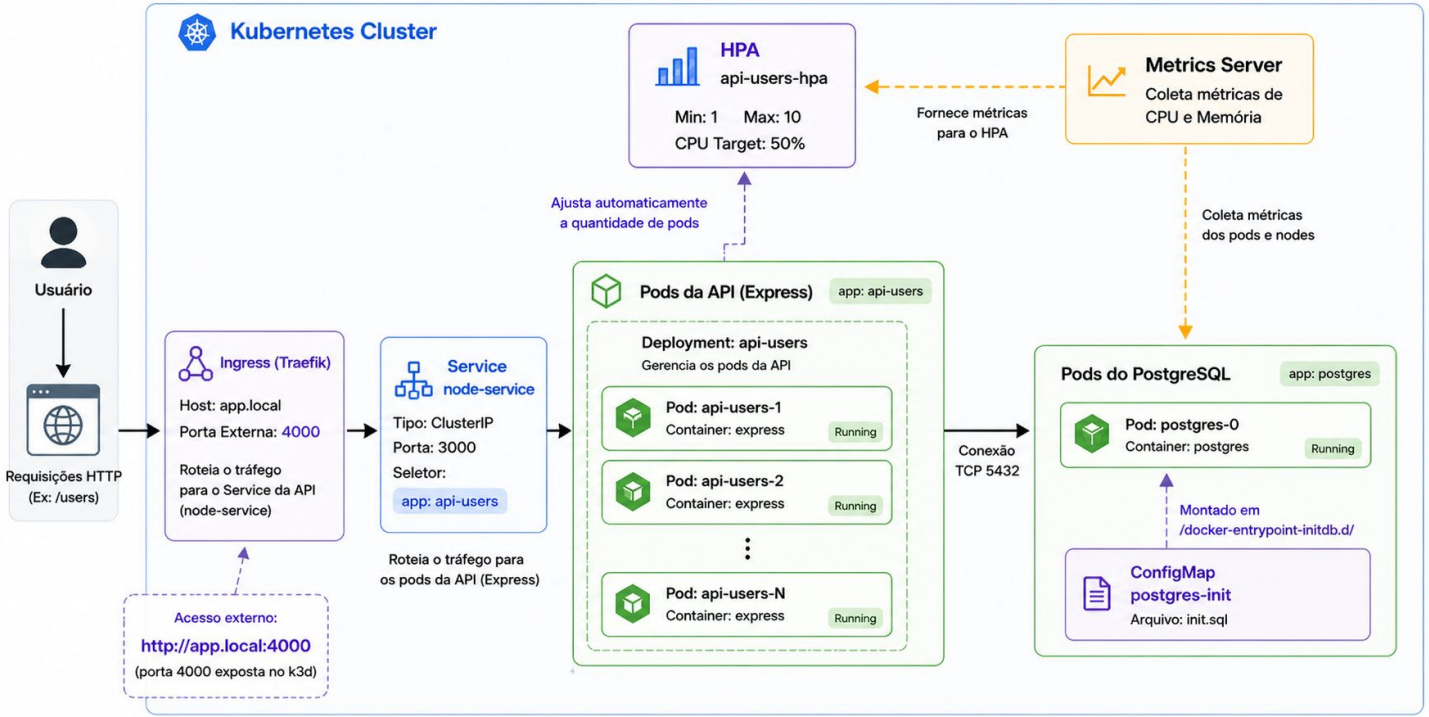
- Secrets
  - Secrets ficam criptografados no etcd
  - Mesmo que alguém acesse o disco

```
apiVersion: apiserver.config.k8s.io/v1
kind: EncryptionConfiguration
resources:
  - resources:
      - secrets
    providers:
      - aescbc:
          keys:
            - name: key1
              secret: <CHAVE_BASE64_AQUI>
      - identity: {}
```

Novos conceitos

- Kubernetes não é só “rodar containers”
- Ele é um orquestrador de recursos
  - Deployment
  - ReplicaSet
  - Service
  - Ingress
  - HPA
  - ConfigMap
  - Metrics Server
  - …

Novos conceitos



Novos conceitos

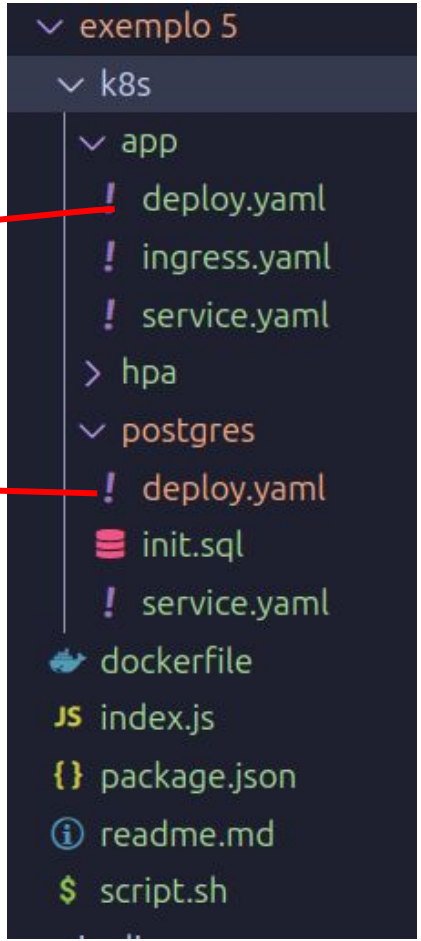
- Deployment
  - Define o estado desejado da aplicação
  - Exemplo: “quero 2 réplicas da API”
  - Kubernetes garante esse estado sempre
  - Responsabilidades:
  - Criar pods
  - Substituir pods com falha
  - Atualizar versões (rolling update)

Novos conceitos

- ReplicaSet
  - Criado automaticamente pelo Deployment
  - Garante número exato de Pods
  - Mantém alta disponibilidade
  - Função:
  - Se um pod morre
    - cria outro
  - Mantém quantidade constante

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api-users
spec:
  replicas: 1
  selector:
    matchLabels:
      app: api-users
  template:
    metadata:
      labels:
        app: api-users
    spec:
      containers:
        - name: api
          image: diegocbcastroo/node-db:1.0
          ports:
            - containerPort: 3000
          env:
            - name: DB_HOST
              value: postgres
            - name: DB_USER
              value: admin
            - name: DB_PASSWORD
              value: admin
            - name: DB_NAME
              value: appdb
          resources:
            requests:
              cpu: "100m"
            limits:
              cpu: "500m"
```

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: postgres
spec:
  replicas: 1
  selector:
    matchLabels:
      app: postgres
  template:
    metadata:
      labels:
        app: postgres
    spec:
      containers:
        - name: postgres
          image: postgres:15
          ports:
            - containerPort: 5432
          env:
            - name: POSTGRES_USER
              value: admin
            - name: POSTGRES_PASSWORD
              value: admin
            - name: POSTGRES_DB
              value: appdb
          volumeMounts:
            - name: init-script
              mountPath: /docker-entrypoint-initdb.d
      volumes:
        - name: init-script
          configMap:
            name: postgres-init
```



```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: postgres
spec:
  replicas: 1
  selector:
    matchLabels:
      app: postgres
  template:
    metadata:
      labels:
        app: postgres
    spec:
      containers:
        - name: postgres
          image: postgres:15
          ports:
            - containerPort: 5432
          env:
            - name: POSTGRES_USER
              value: admin
            - name: POSTGRES_PASSWORD
              value: admin
            - name: POSTGRES_DB
              value: appdb
          volumeMounts:
            - name: init-script
              mountPath: /docker-entrypoint-initdb.d
      volumes:
        - name: init-script
          configMap:
            name: postgres-init
```

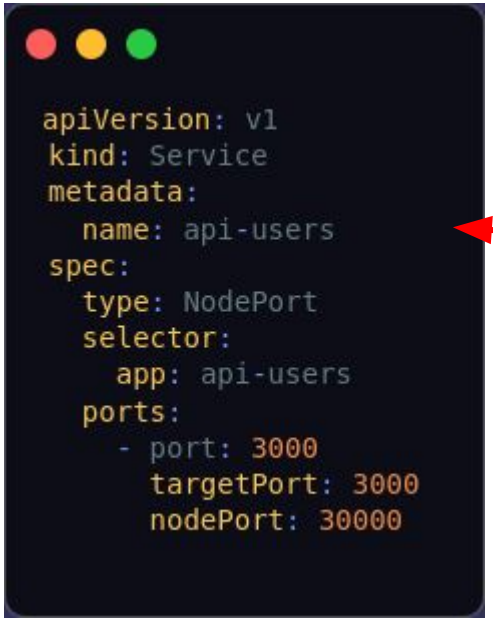
Diretório para colocar o init.sql

```
kubectl create configmap postgres-init \
  --from-file=k8s/postgres/init.sql
```



Novos conceitos

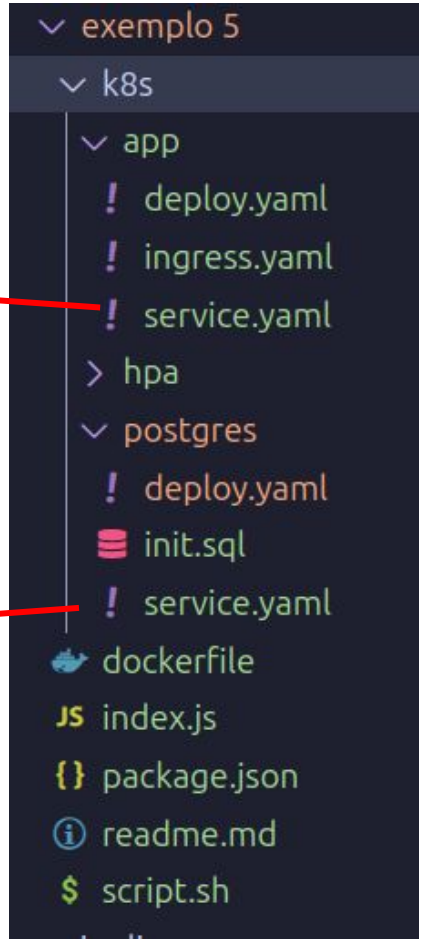
- Service é o componente que:
  - Expõe e conecta aplicações dentro (ou fora) do cluster
  - Problema que ele resolve
  - Pods:
    - mudam de IP o tempo todo
    - podem morrer e recriar
    - não são estáveis



```yaml
apiVersion: v1
kind: Service
metadata:
  name: api-users
spec:
  type: NodePort
  selector:
    app: api-users
  ports:
    - port: 3000
      targetPort: 3000
      nodePort: 30000
```



```yaml
apiVersion: v1
kind: Service
metadata:
  name: postgres
spec:
  type: ClusterIP
  selector:
    app: postgres
  ports:
    - port: 5432
      targetPort: 5432
```



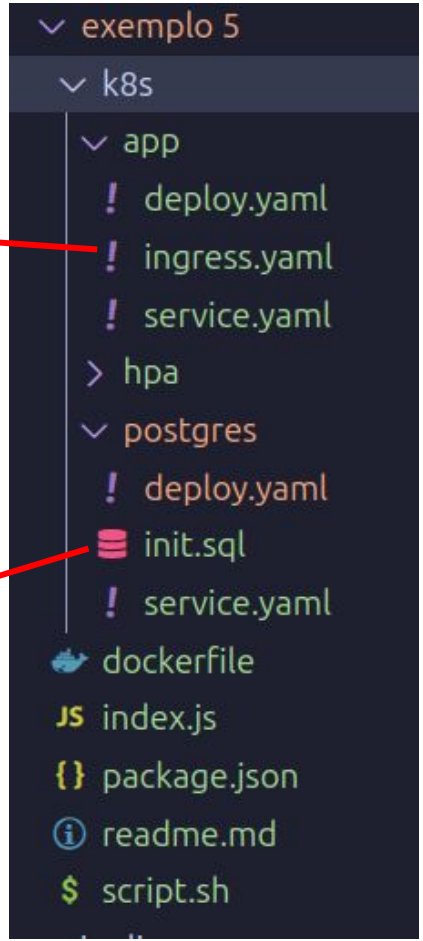
Novos conceitos

- Ingress é a camada que:
  - Expõe aplicações HTTP/HTTPS para fora do cluster
  - Ele funciona como um roteador inteligente (reverse proxy)
  - Sem Ingress:
    - você usaria NodePort (porta feia tipo 31234)
    - ou LoadBalancer por serviço
  - Com Ingress:
    - URL bonita
    - roteamento centralizado
    - HTTPS
    - múltiplas apps no mesmo IP

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: node-ingress
spec:
  rules:
  - http:
      paths:
        - path: /
          pathType: Prefix
          backend:
            service:
              name: api-users
              port:
                number: 3000
```

```sql
CREATE TABLE users (
  id SERIAL PRIMARY KEY,
  name TEXT
);

INSERT INTO users (name)
SELECT 'User ' || generate_series(1,1000);
```



Novos conceitos

- Autoscaling automático
  - Escala Pods automaticamente
  - Baseado em métricas (CPU/memória)
  - Ajusta o ReplicaSet dinamicamente
  - Regras:
    - minReplicas
    - maxReplicas
    - target CPU %
    - …

Novos conceitos

- O HPA pode escalar baseado em 4 categorias
  - Resource Metrics
    - cpu
    - memory
  - Pod Metrics
    - requests por segundo
    - filas por pod
    - latência por pod

```yaml
type: Resource
resource:
  name: cpu
  target:
    type: Utilization
    averageUtilization: 70
```

```yaml
type: Pods
pods:
  metric:
    name: requests_per_second
  target:
    type: AverageValue
    averageValue: "100"
```

Novos conceitos

- O HPA pode escalar baseado em 4 categorias
  - Object Metrics
    - Número de mensagens em uma fila SQS/RabbitMQ
  - External Metrics
    - Tráfego HTTP no API Gateway

```yaml
type: Object
object:
  metric:
    name: queue_messages
  describedObject:
    kind: Service
    name: rabbitmq
  target:
    type: Value
    value: "200"
```

```yaml
type: External
external:
  metric:
    name: sqs_messages_visible
  target:
    type: Value
    value: "300"
```

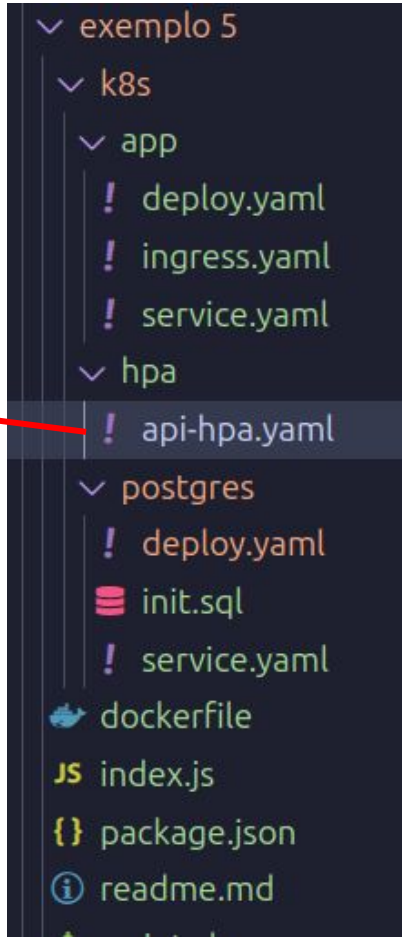
Nunca terá menos que 1 e no máximo 10

- Comportamento de escala
  - Espera 30 segundos antes de reduzir pods
  - Redução em porcentagem
  - Reduzir até 100% dos pods extras
  - Avalia a cada 15 segundos

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: api-users-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: api-users
  minReplicas: 1
  maxReplicas: 10

  behavior:
    scaleDown:
      stabilizationWindowSeconds: 30
      policies:
        - type: Percent
          value: 100
          periodSeconds: 15

  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 50
```

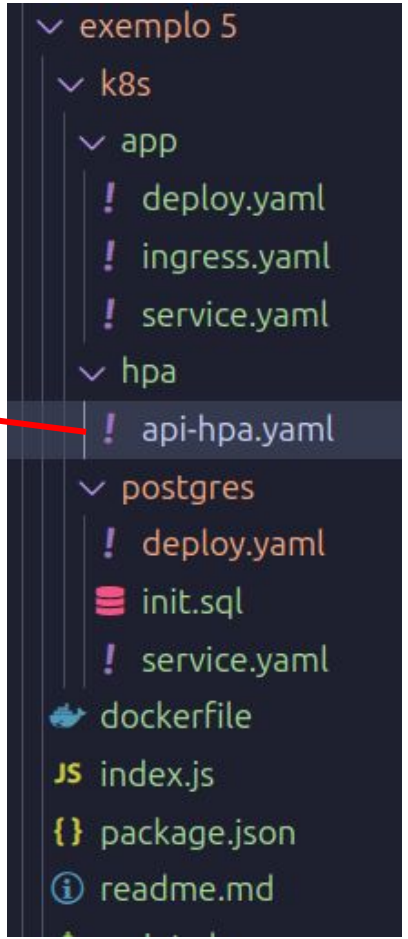


- Métricas do próprio Kubernetes
  - CPU dos pods
  - Se a média de CPU dos pods passar de 50%, o HPA escala para cima

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: api-users-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: api-users
  minReplicas: 1
  maxReplicas: 10

  behavior:
    scaleDown:
      stabilizationWindowSeconds: 30
      policies:
        - type: Percent
          value: 100
          periodSeconds: 15

  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 50
```



Novos conceitos

- Coleta de métricas
  - Componente que fornece dados ao HPA
  - Sem ele
    - autoscaling não funciona
  - Ele coleta
    - CPU usage
    - Memória
    - Uso por pod/node

Novos conceitos

```
k3d kubeconfig merge meucluster

kubectl delete -f k8s/hpa/ --ignore-not-found
kubectl delete -f k8s/app/ --ignore-not-found
kubectl delete -f k8s/postgres/ --ignore-not-found

kubectl delete configmap postgres-init --ignore-not-found

kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml --validate=false

sleep 5

kubectl patch deployment metrics-server -n kube-system \
  --type='json' \
  -p='[
    {"op":"add","path":"/spec/template/spec/containers/0/args/-","value":"--kubelet-insecure-tls"},
    {"op":"add","path":"/spec/template/spec/containers/0/args/-","value":"--kubelet-preferred-address-types=InternalIP"}
  ]'
```

Novos conceitos

```
kubectl create configmap postgres-init \
  --from-file=k8s/postgres/init.sql

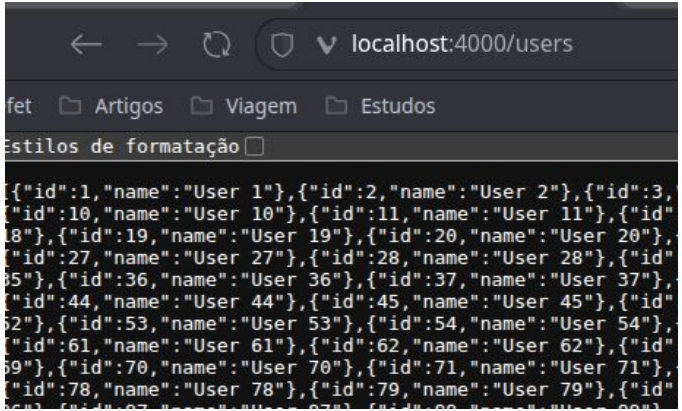

kubectl apply -f k8s/app/service.yaml
kubectl apply -f k8s/app/deploy.yaml
kubectl apply -f k8s/app/ingress.yaml

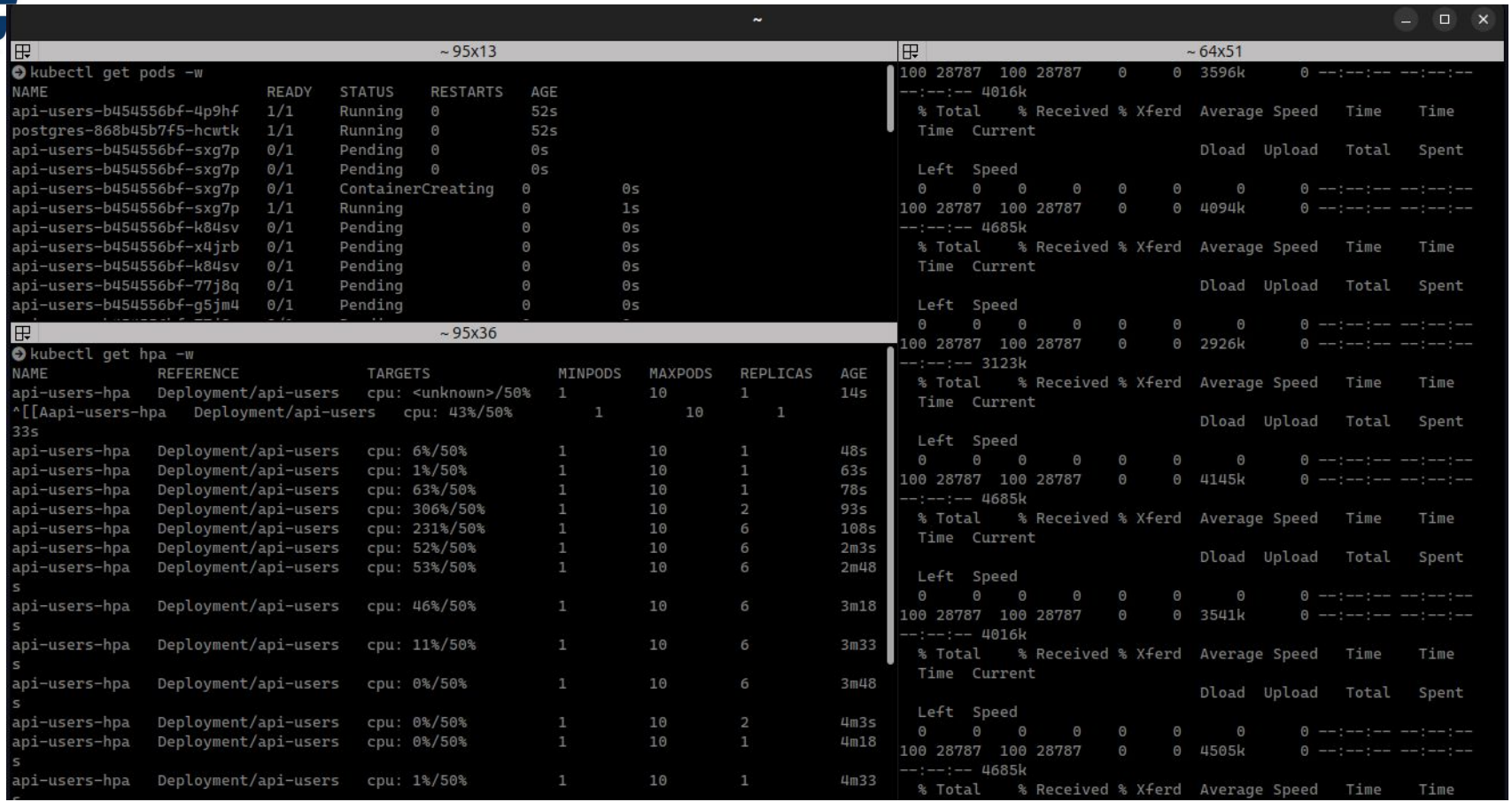
kubectl apply -f k8s/postgres/service.yaml
kubectl apply -f k8s/postgres/deploy.yaml

kubectl apply -f k8s/hpa/

kubectl wait --for=condition=available deployment/postgres --timeout=60s
kubectl wait --for=condition=available deployment/api-users --timeout=60s

kubectl get pods
kubectl get svc
kubectl get hpa
kubectl get ingress
```




~
~ 95x13
kubectl get pods -w
NAME READY STATUS RESTARTS AGE
api-users-b454556bf-4p9hf 1/1 Running 0 52s
postgres-868b45b7f5-hcwtk 1/1 Running 0 52s
api-users-b454556bf-sxg7p 0/1 Pending 0 0s
api-users-b454556bf-sxg7p 0/1 Pending 0 0s
api-users-b454556bf-sxg7p 0/1 ContainerCreating 0 0s
api-users-b454556bf-sxg7p 1/1 Running 0 1s
api-users-b454556bf-k84sv 0/1 Pending 0 0s
api-users-b454556bf-x4jrb 0/1 Pending 0 0s
api-users-b454556bf-k84sv 0/1 Pending 0 0s
api-users-b454556bf-77j8q 0/1 Pending 0 0s
api-users-b454556bf-g5jm4 0/1 Pending 0 0s
~ 95x36
kubectl get hpa -w
NAME REFERENCE TARGETS MINPODS MAXPODS REPLICAS AGE
api-users-hpa Deployment/api-users cpu: <unknown>/50% 1 10 1 14s
^[[Aapi-users-hpa Deployment/api-users cpu: 43%/50% 1 10 1
33s
api-users-hpa Deployment/api-users cpu: 6%/50% 1 10 1 48s
api-users-hpa Deployment/api-users cpu: 1%/50% 1 10 1 63s
api-users-hpa Deployment/api-users cpu: 63%/50% 1 10 1 78s
api-users-hpa Deployment/api-users cpu: 306%/50% 1 10 2 93s
api-users-hpa Deployment/api-users cpu: 231%/50% 1 10 6 108s
api-users-hpa Deployment/api-users cpu: 52%/50% 1 10 6 2m3s
api-users-hpa Deployment/api-users cpu: 53%/50% 1 10 6 2m48
s
api-users-hpa Deployment/api-users cpu: 46%/50% 1 10 6 3m18
s
api-users-hpa Deployment/api-users cpu: 11%/50% 1 10 6 3m33
s
api-users-hpa Deployment/api-users cpu: 0%/50% 1 10 6 3m48
s
api-users-hpa Deployment/api-users cpu: 0%/50% 1 10 2 4m3s
api-users-hpa Deployment/api-users cpu: 0%/50% 1 10 1 4m18
s
api-users-hpa Deployment/api-users cpu: 1%/50% 1 10 1 4m33
~ 64x51
100 28787 100 28787 0 0 3596k 0 --:--:-- --:--:--
--:--:-- 4016k
% Total % Received % Xferd Average Speed Time Time
Time Current
Dload Upload Total Spent
Left Speed
0 0 0 0 0 0 0 0 --:--:-- --:--:--
100 28787 100 28787 0 0 4094k 0 --:--:-- --:--:--
--:--:-- 4685k
% Total % Received % Xferd Average Speed Time Time
Time Current
Dload Upload Total Spent
Left Speed
0 0 0 0 0 0 0 0 --:--:-- --:--:--
100 28787 100 28787 0 0 2926k 0 --:--:-- --:--:--
--:--:-- 3123k
% Total % Received % Xferd Average Speed Time Time
Time Current
Dload Upload Total Spent
Left Speed
0 0 0 0 0 0 0 0 --:--:-- --:--:--
100 28787 100 28787 0 0 4145k 0 --:--:-- --:--:--
--:--:-- 4685k
% Total % Received % Xferd Average Speed Time Time
Time Current
Dload Upload Total Spent
Left Speed
0 0 0 0 0 0 0 0 --:--:-- --:--:--
100 28787 100 28787 0 0 3541k 0 --:--:-- --:--:--
--:--:-- 4016k
% Total % Received % Xferd Average Speed Time Time
Time Current
Dload Upload Total Spent
Left Speed
0 0 0 0 0 0 0 0 --:--:-- --:--:--
100 28787 100 28787 0 0 4505k 0 --:--:-- --:--:--
--:--:-- 4685k
% Total % Received % Xferd Average Speed Time Time

Mais recursos

- imagePullPolicy
  - Always
  - IfNotPresent
  - Never
  - Regra
    - Se a imagem tiver :latest
      - Always
    - Se tiver tag específica
      - IfNotPresent

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api-exemplo
spec:
  replicas: 2
  selector:
    matchLabels:
      app: api-exemplo
  template:
    metadata:
      labels:
        app: api-exemplo
    spec:
      containers:
        - name: api
          image: nginx:1.25
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 80
```

Mais recursos

- Restart Policy
  - Define o comportamento do container quando ele falha ou termina
  - Always
  - OnFailure
  - Never



```yaml
apiVersion: v1
kind: Pod
metadata:
  name: restart-policy-exemplo
spec:
  restartPolicy: Always
  containers:
    - name: app
      image: nginx:1.25
      ports:
        - containerPort: 80
```

Mais recursos

- Command e Args
  - Permite sobrescrever o comportamento padrão da imagem Docker
  - Alterar o que o container executa sem precisar criar uma nova imagem

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: command-args-exemplo
spec:
  containers:
    - name: app
      image: nginx:1.25
      command: ["/bin/sh", "-c"]
      args: ["echo Hello Kubernetes && sleep 3600"]
```

Self Healing

- Self Healing
  - Capacidade do Kubernetes de se recuperar automaticamente de falhas
  - Garantir que o sistema continue funcionando mesmo com falhas em containers
    - ReplicaSet / Deployment
    - Liveness Probe
    - Readiness Probe
    - Startup Probe

Self Healing

- Liveness Probe
  - Verifica se o container ainda está vivo
  - Se falhar
    - Kubernetes reinicia o container
  - Tipos
    - HTTP: endpoint de saúde
    - Exec: comando interno
    - TCP: valida porta aberta

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: liveness-exemplo
spec:
  containers:
    - name: app
      image: nginx:1.25
      ports:
        - containerPort: 80
      livenessProbe:
        httpGet:
          path: /
          port: 80
        initialDelaySeconds: 5
        periodSeconds: 10
```

Self Healing

- Readiness Probe
  - Verifica se o container está pronto para receber tráfego.
  - Se falhar
    - Kubernetes remove o pod do Service temporariamente

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: readiness-exemplo
spec:
  containers:
    - name: app
      image: nginx:1.25
      ports:
        - containerPort: 80
      readinessProbe:
        tcpSocket:
          port: 80
        initialDelaySeconds: 5
        periodSeconds: 10
```

Self Healing

- Startup Probe
  - Evita que o Kubernetes mate aplicações lentas no startup

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: startup-exemplo
spec:
  containers:
    - name: app
      image: nginx:1.25
      ports:
        - containerPort: 80
      startupProbe:
        httpGet:
          path: /
          port: 80
        failureThreshold: 30
        periodSeconds: 5
```

Self Healing

